Semantic Classification Assignment Report: Fake News Detection

- **Submitted by:**
  Tejashri Pilla & Deepajothi S K
  ML C72 (EPGP) December 2024

- **Submitted to:**
  upGrad and IIITB Machine Learning & AI Program

- **Date:**
  September 25, 2025

- **Course:**
  Machine Learning & AI

- **Institution:**
  IIIT Bangalore

Contents

1. Data Preparation 7
   1.0. Data Understanding 7
   1.1. Add new column 7
   1.2. Merge DataFrames 7
   1.3. Handle the null values 7
   1.4. Merge the relevant columns and drop the rest from the DataFrame 8
2. Text Preprocessing 8
   2.1. Text Cleaning 8
      2.1.0. Create a new DataFrame to store the processed data 8
      2.1.1. Write the function to clean the text and remove all the unnecessary elements .. 8
      2.1.2. Apply the function to clean the news text and store the cleaned text in a new column within the new DataFrame 8
   2.2. POS Tagging and Lemmatization 9
      2.2.1 Write the function for POS tagging and lemmatization, filtering stopwords and keeping only NN and NNS tags 9
      2.2.2 Apply the POS tagging and lemmatization function to cleaned text and store it in a new column within the new DataFrame. 9
   2.3. Save the Cleaned data as a csv file 9
3. Train Validation Split 10
4. Exploratory Data Analysis on Training Data 10
   4.1 Visualise character lengths of cleaned news text and lemmatized news text with POS tags removed 10
      4.1.1 Add new columns to calculate the character lengths of the processed data columns 10
      4.1.2 Create Histogram to visualise character lengths 11
   4.2 Find and display the top 40 words by frequency among true and fake news in Training data after processing the text 11
      4.2.1 Find and display the top 40 words by frequency among true news in Training data after processing the text 11
      4.2.2 Find and display the top 40 words by frequency among fake news in Training data after processing the text 12
   4.3 Find and display the top unigrams, bigrams and trigrams by frequency in true news and fake news after processing the text 13
      4.3.1 Write a function to get the specified top n-grams 13

4.3.2 Handle the NaN values 13
4.3.3 Display the top 10 unigrams by frequency in true news and plot them as a bar graph 13
4.3.4 Display the top 10 bigrams by frequency in true news and plot them as a bar graph 14
4.3.5 Display the top 10 trigrams by frequency in true news and plot them as a bar graph 14
4.3.6 Display the top 10 unigrams by frequency in fake news and plot them as a bar graph 15
4.3.7 Display the top 10 bigrams by frequency in fake news and plot them as a bar graph 15
4.3.8 Display the top 10 trigrams by frequency in fake news and plot them as a bar graph 16
5. Exploratory Data Analysis on Validation Data.....16
5.1 Visualise character lengths of cleaned news text and lemmatized news text with POS tags removed.....16
5.1.1 Add new columns to calculate the character lengths of the processed data columns 16
5.1.2 Create Histogram to visualise character lengths.....17
5.2 Find and display the top 40 words by frequency among true and fake news after processing the text.....17
5.2.1 Find and display the top 40 words by frequency among true news in validation data after processing the text 17
5.2.2 Find and display the top 40 words by frequency among fake news in validation data after processing the text 18
5.3 Find and display the top unigrams, bigrams and trigrams by frequency in true news and fake news after processing the text 19
5.3.1 Write a function to get the specified top n-grams.....19
5.3.2 Handle the NaN values 19
5.3.3 Display the top 10 unigrams by frequency in true news and plot them as a bar graph 19
5.3.4 Display the top 10 bigrams by frequency in true news and plot them as a bar graph 20
5.3.5 Display the top 10 trigrams by frequency in true news and plot them as a bar graph 20
5.3.6 Display the top 10 unigrams by frequency in fake news and plot them as a bar graph 21

5.3.7 Display the top 10 bigrams by frequency in fake news and plot them as a bar graph 21

5.3.8 Display the top 10 trigrams by frequency in fake news and plot them as a bar graph 22

6. Feature Extraction 22

6.1 Initialise Word2Vec model 22

6.2 Extract vectors for cleaned news data 22

7. Model Training and Evaluation 23

7.0 Import models and evaluation metrics 23

7.1 Build Logistic Regression Model.....23

7.1.1. Create and train logistic regression model on training data 23

7.1.2. Calculate and print accuracy, precision, recall and f1-score on validation data.23

7.2 Build Decision Tree Model.....24

7.2.1 Create and train a decision tree model on training data.....24

7.2.2 Calculate and print accuracy, precision, recall and f1-score on validation data.24

7.3 Build Random Forest Model 25

7.3.1 Create and train a random forest model on training data.....25

7.3.2 Calculate and print accuracy, precision, recall and f1-score on validation data.25

8. Conclusion 26

- ## Objective

The objective of this assignment is to develop a Semantic Classification model. You will be using Word2Vec method to extract the semantic relations from the text and develop a basic understanding of how to train supervised models to categorise text based on its meaning, rather than just syntax. You will explore how this technique is used in situations where understanding textual meaning plays a critical role in making accurate and efficient decisions.

- ## Business Objective

The spread of fake news has become a significant challenge in today's digital world. With the massive volume of news articles published daily, it's becoming harder to distinguish between credible and misleading information. This creates a need for systems that can automatically classify news articles as true or fake, helping to reduce misinformation and protect public trust.
In this assignment, you will develop a Semantic Classification model that uses the Word2Vec method to detect recurring patterns and themes in news articles. Using supervised learning models, the goal is to build a system that classifies news articles as either fake or true.

- ## Data Dictionary

For this assignment, we will work with two datasets, **True.csv** and **Fake.csv**. Both datasets contain three columns:

- Title of the news article
- Text of the news article
- Date of article publication

True.csv dataset includes 21,417 true news, while the **Fake.csv** dataset comprises 23,502 fake news.

- ## Installing required Libraries

For this assignment, we have installed required libraries like:

- Numpy
- Pandas
- NLTK
- Spacy
- Scipy
- Pydantic
- Wordcloud etc.

- ## Importing the necessary libraries

- We have imported essential libraries for data manipulation and analysis.
- We have loaded the spaCy small English language model.
- We have imported libraries for data visualization.
- We have imported libraries for interactive plots.
- We have imported libraries for pre-processing and feature extraction in machine learning.
- We have imported accuracy, precision, recall, f_score from sklearn to predict train accuracy.
- We have suppressed unnecessary warnings to keep output clean.
- We have configured Matplotlib to display plots inline in Jupyter Notebook.

- ## Load the data

- We have loaded the data from both the csv files provided.
- We have created DataFrame true_df by loading True.csv file and DataFrame fake_df by loading Fake.csv file.

1. Data Preparation

1.0. Data Understanding

- We have inspected the DataFrame with True News to understand the given data.
- Similarly we have inspected the DataFrame with Fake News to understand the given data.
- Also we have checked the column details for True News DataFrame and Fake News DataFrame.
- Both the DataFrame has 3 columns as 'title', 'text', 'date'.

1.1. Add new column

- We have added new column as ‘news_label’ to both the DataFrames.
- We have added a new column 'news_label' to the true news DataFrame and assign the label "1" to indicate that these news are true.
- We have added a new column 'news_label' to the fake news DataFrame and assign the label "0" to indicate that these news are fake.

1.2. Merge DataFrames

- We have combined the true and fake news DataFrames into a single DataFrame called news_df.
- And we have checked some rows of this combined DataFrame to verify the result.
- Also we have checked shape of combined DataFrame to check whether it contain all rows from both the DataFrames.
- We have confirmed if both the data sets are merged properly or not by checking value counts for news labels.

1.3. Handle the null values

- We have checked for null values in merged DataFrame.
- As we can see that there are **21** null values in **title, text** and **42** null values in **date**.
- These null values are less than 0.1% of total data.
- Hence, we can remove these nulls as they don't provide any useful information to us.

1.4. Merge the relevant columns and drop the rest from the DataFrame

- We have combined the ‘title’ and ‘text’ columns into a new column 'news_text' by joining their values with a space.
- Since we have joined the ‘title’ and ‘text’ together, we now no need these columns.
- Also ‘Date’ column does not help us anything in prediction of True news or Fake news.
- Hence we will drop these columns. (‘title’, ‘text’, ‘date’)

2. Text Preprocessing

2.1. Text Cleaning

2.1.0. Create a new DataFrame to store the processed data

- We have created a DataFrame ('df_clean') that will have only the cleaned news text and the lemmatized news text with POS tags removed.
- For now we have just created an empty DataFrame 'df_clean'.
- We have added 'news_label' column same as news_df to the new DataFrame for topic identification.

2.1.1. Write the function to clean the text and remove all the unnecessary elements

- We have created a function ‘text_cleanup’ which will perform below tasks:
    - Convert the text to lower case
    - Remove the text present in square bracket
    - Remove Punctuation
    - Remove words with numbers

2.1.2. Apply the function to clean the news text and store the cleaned text in a new column within the new DataFrame.

- We have applied this ‘text_cleanup’ function to clean the news text and remove all unnecessary elements.
- We have stored this cleaned text in **cleaned_text column** of df_clean DataFrame.

```
df_clean.head()
```

	news_label	cleaned_text
0	1	as us budget fight looms republicans flip thei...
1	1	us military to accept transgender recruits on ...
2	1	senior us republican senator let mr mueller do...
3	1	fbi russia probe helped by australian diplomat...
4	1	trump wants postal service to charge much more...

2.2. POS Tagging and Lemmatization

2.2.1 Write the function for POS tagging and lemmatization, filtering stopwords and keeping only NN and NNS tags

- We have created the function for POS tagging and lemmatization, filtering stopwords and keeping only NN and NNS tags.
- This function will return list of lemmatized words.

2.2.2 Apply the POS tagging and lemmatization function to cleaned text and store it in a new column within the new DataFrame.

- We have applied POS tagging and lemmatization function to cleaned text and stored the lemmatized text in new column in the df_clean DataFrame.
- This will be useful for analysing character length differences between cleaned text and lemmatized text with POS tags removed during EDA.
- Until now the data is true news followed by Fake news which may give issues in training the data, hence we have shuffled the dataset.

2.3. Save the Cleaned data as a csv file

- It is taking time to apply POS tagging and lemmatize the data, hence we have saved the cleaned data to csv file.
- And then next time whenever you are working on this, you can load csv file to save time while rerunning the code.
- After loading the data from csv file, we have checked first few rows, dimensions of the DataFrame, number of non-null entries and data types of each column.

3. Train Validation Split

- We have imported train_test_split from sklearn.model_selection and split the DataFrame into 70% train and 30% validation data.
- After splitting we have got X_train, X_val, y_train, y_val, we have combined X_train and y_train to create train_df and combined X_val and y_val to create val_df for future analysis.
- We have checked dimensions for train_df and val_df.

4. Exploratory Data Analysis on Training Data

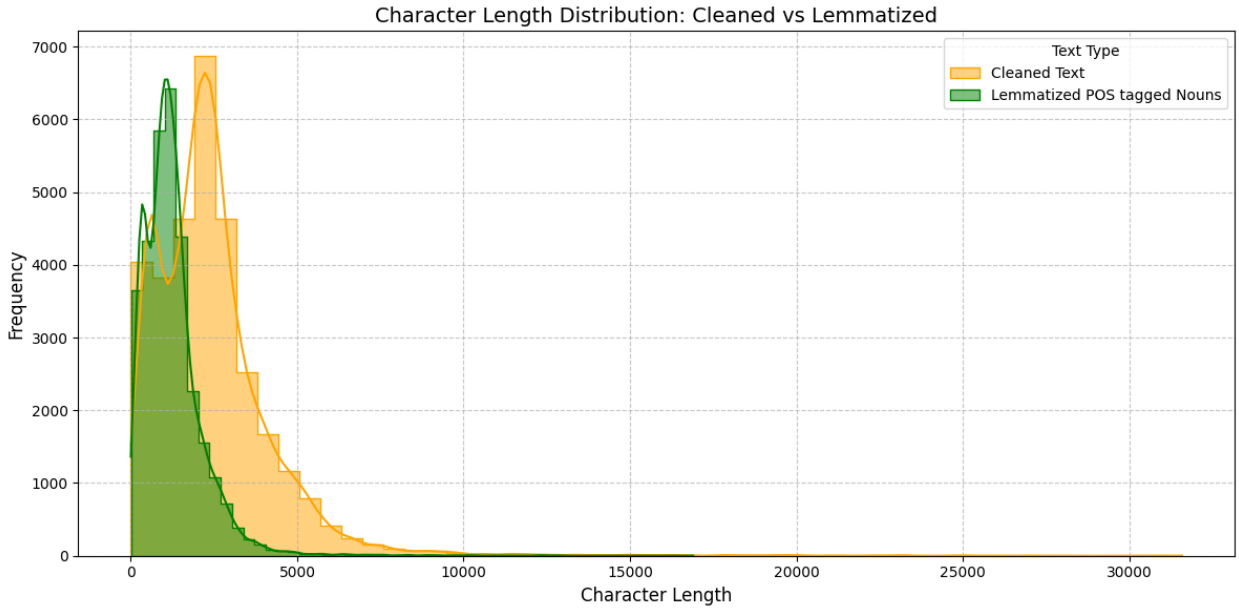
4.1 Visualise character lengths of cleaned news text and lemmatized news text with POS tags removed

4.1.1 Add new columns to calculate the character lengths of the processed data columns

- We have added a new column ‘cleaned_text_len’ to train_df to calculate and store the character length of cleaned news text.
- We have also added another column ‘lemmatized_text_len’ to train_df to calculate the character length of lemmatized news text with POS tags removed.

4.1.2 Create Histogram to visualise character lengths

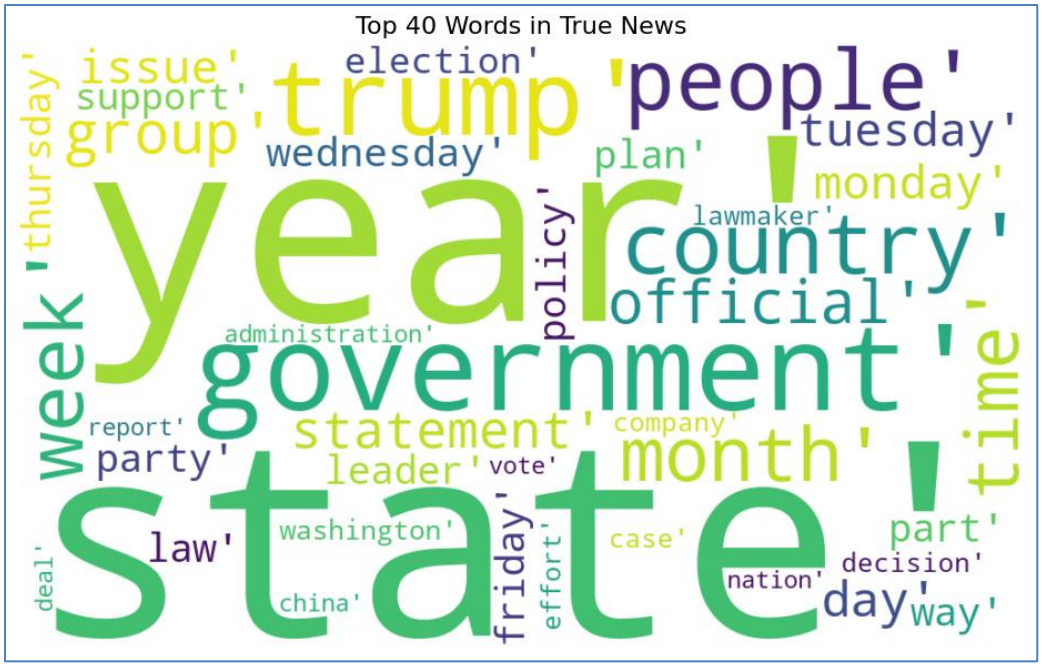
- We have created histogram plot to visualise character lengths.
- We have plotted both distributions on the same graph for comparison and to observe overlaps and peak differences to understand text pre-processing's impact on text length.



4.2 Find and display the top 40 words by frequency among true and fake news in Training data after processing the text

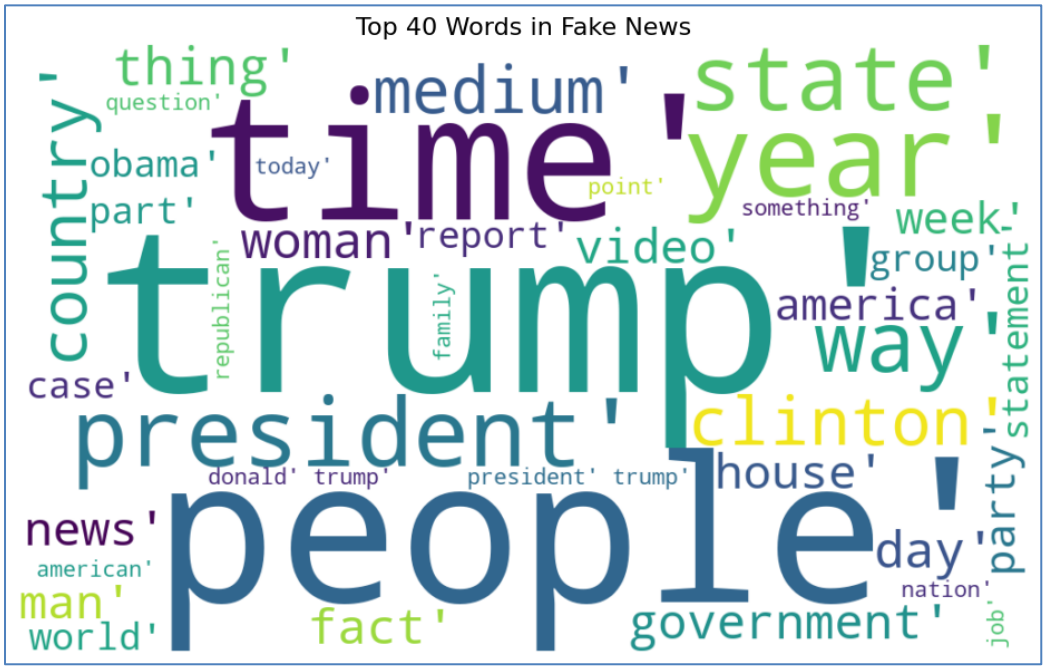
4.2.1 Find and display the top 40 words by frequency among true news in Training data after processing the text

- We have used a word cloud to find the top 40 words by frequency among true news in the training data after processing the text.
- We have filtered news with label 1 (True News) and convert to it string and handle any non-string values.
- And then generated word cloud for True News.



4.2.2 Find and display the top 40 words by frequency among fake news in Training data after processing the text

- Similarly, we have used a word cloud to find the top 40 words by frequency among fake news in the training data after processing the text.
- We have filtered news with label 0 (Fake News) and convert to it string and handle any non-string values.
- And then generated word cloud for Fake News.



4.3 Find and display the top unigrams, bigrams and trigrams by frequency in true news and fake news after processing the text

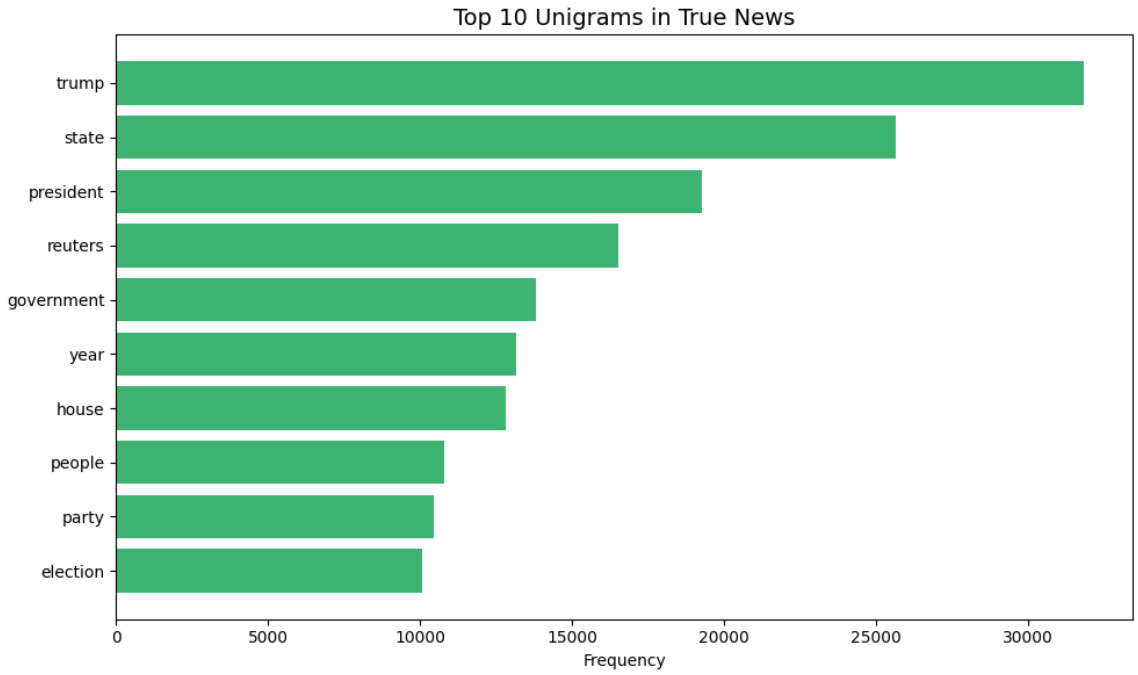
4.3.1 Write a function to get the specified top n-grams

- We have defined a function to get the specified top n-grams.
- This function takes corpus, n-gram range, how many top grams you need, title of the n-gram and color of the n-gram as input parameters.
- And plots n-gram for this.

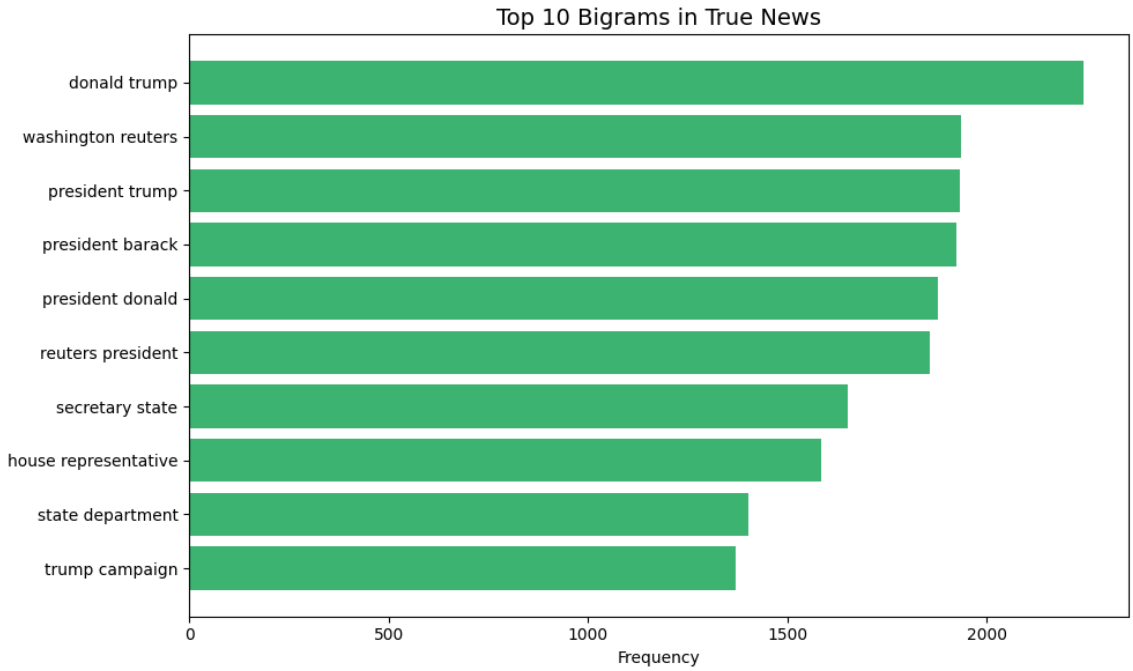
4.3.2 Handle the NaN values

- There are no NaN values present in train dataframe as we have already handled them earlier.

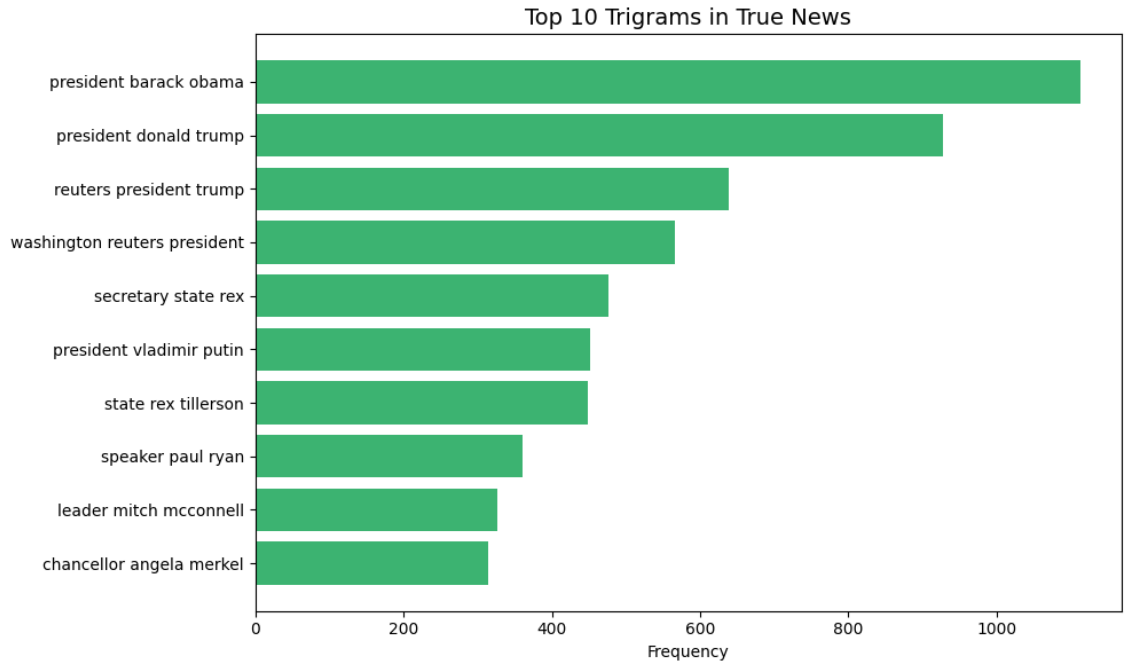
4.3.3 Display the top 10 unigrams by frequency in true news and plot them as a bar graph



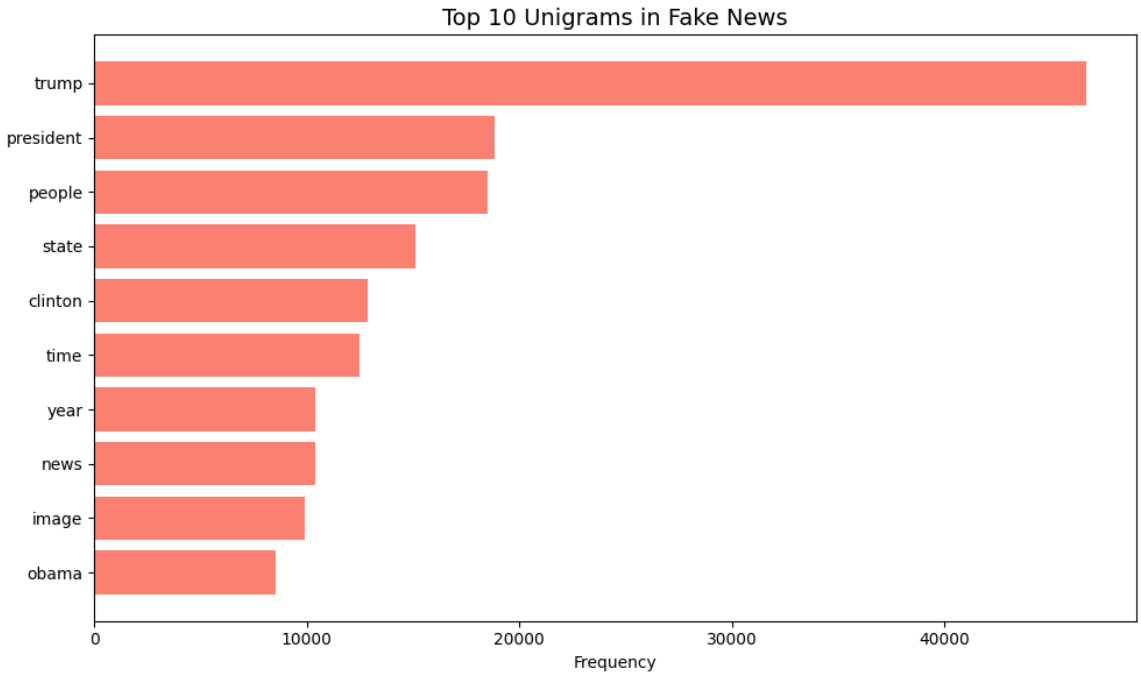
4.3.4 Display the top 10 bigrams by frequency in true news and plot them as a bar graph



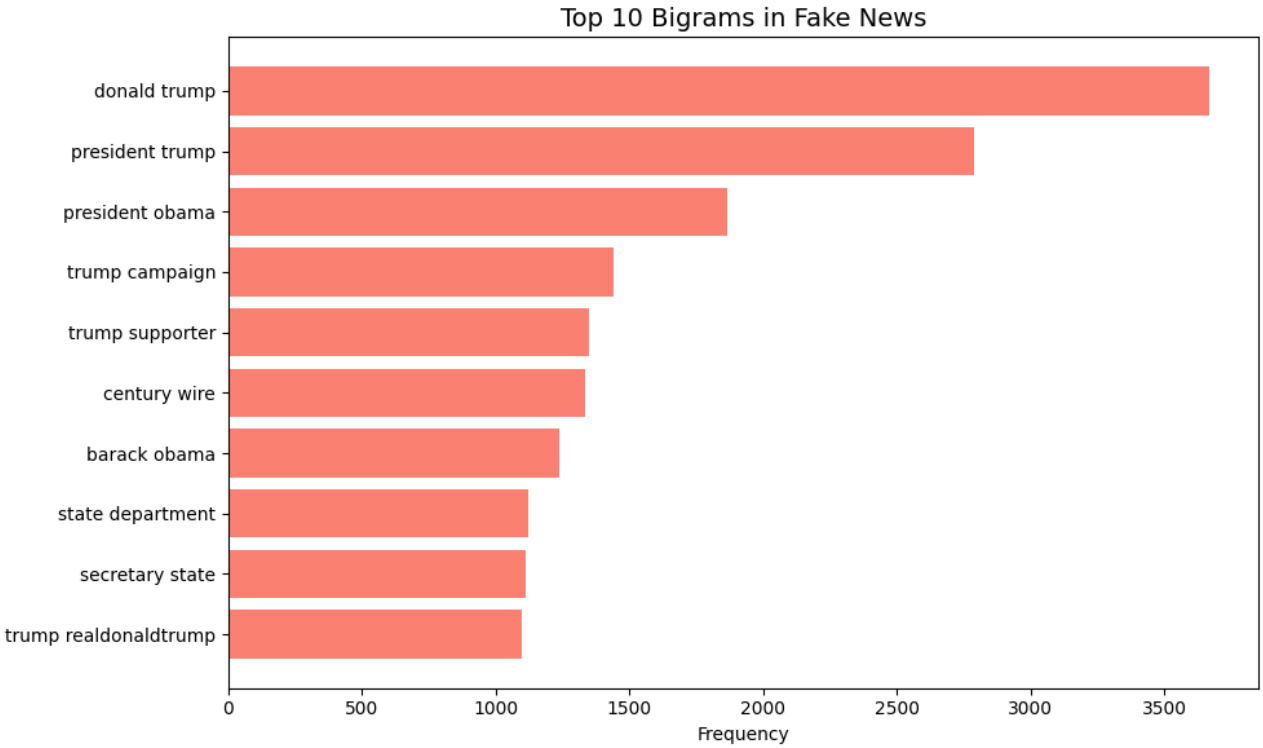
4.3.5 Display the top 10 trigrams by frequency in true news and plot them as a bar graph



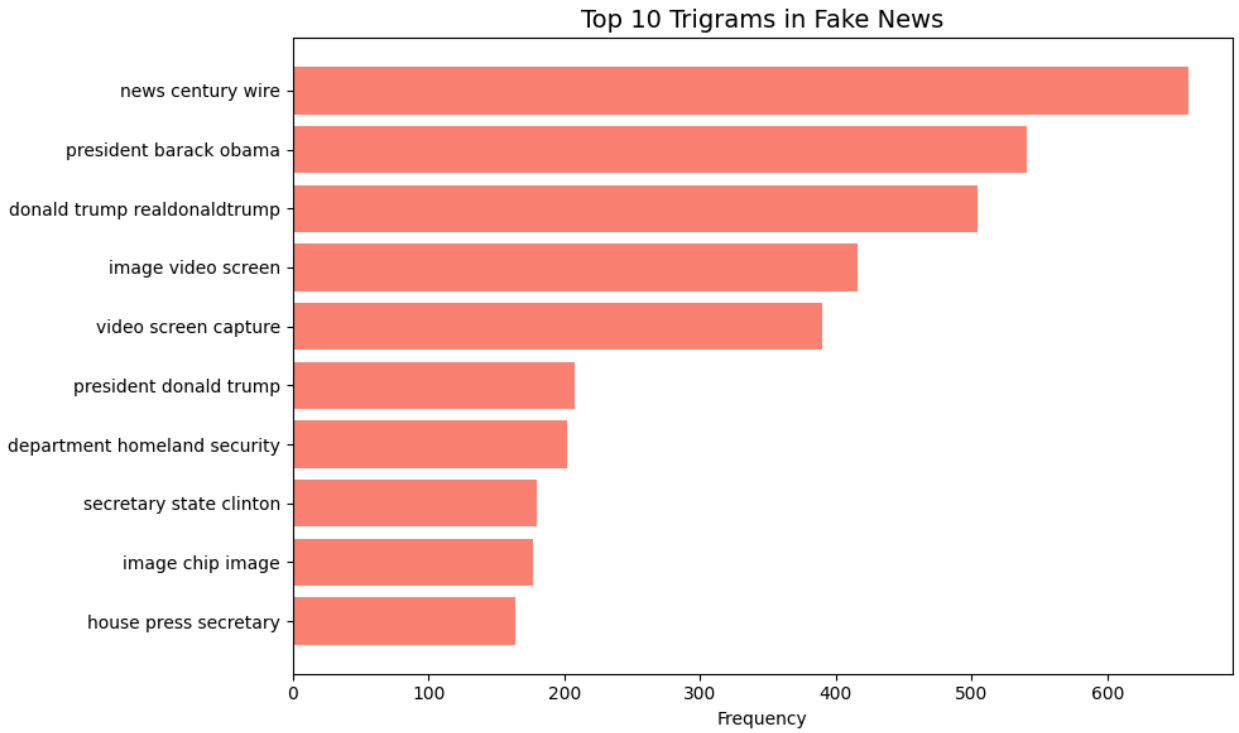
4.3.6 Display the top 10 unigrams by frequency in fake news and plot them as a bar graph



4.3.7 Display the top 10 bigrams by frequency in fake news and plot them as a bar graph



4.3.8 Display the top 10 trigrams by frequency in fake news and plot them as a bar graph



5. Exploratory Data Analysis on Validation Data

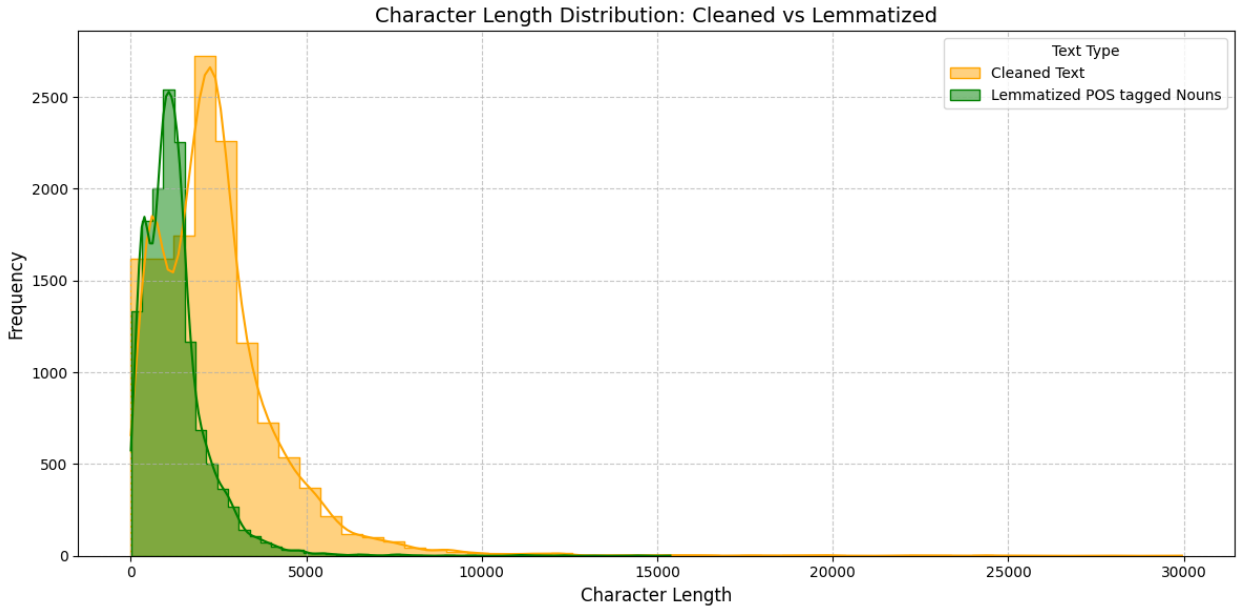
5.1 Visualise character lengths of cleaned news text and lemmatized news text with POS tags removed

5.1.1 Add new columns to calculate the character lengths of the processed data columns

- We have added a new column ‘cleaned_text_len’ to val_df to calculate and store the character length of cleaned news text.
- We have also added another column ‘lemmatized_text_len’ to val_df to calculate the character length of lemmatized news text with POS tags removed.

5.1.2 Create Histogram to visualise character lengths

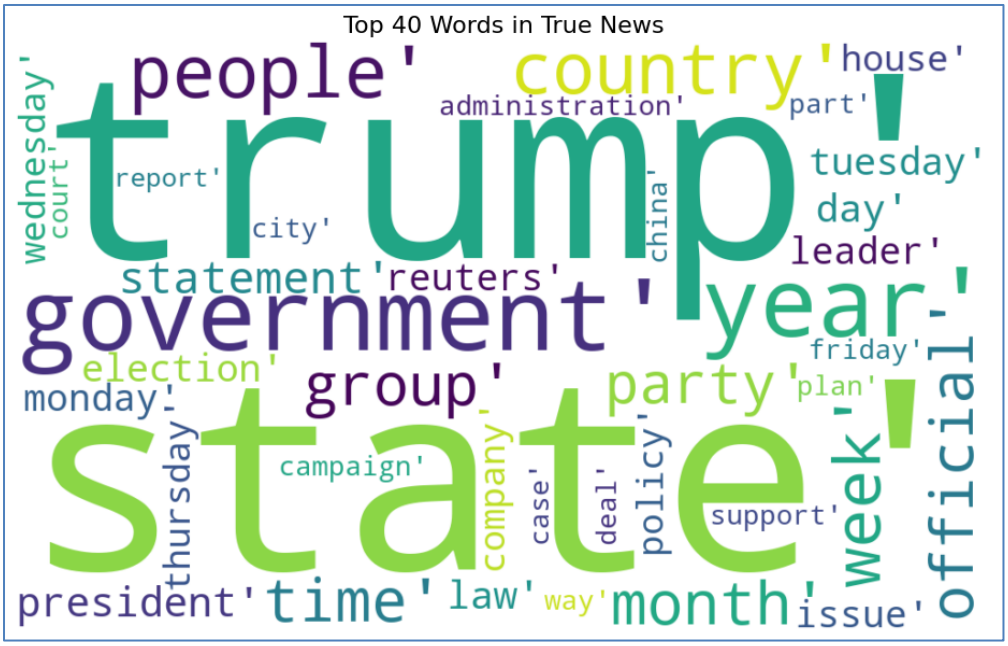
- We have created histogram plot to visualise character lengths.
- We have plotted both distributions on the same graph for comparison and to observe overlaps and peak differences to understand text pre-processing's impact on text length.



5.2 Find and display the top 40 words by frequency among true and fake news after processing the text

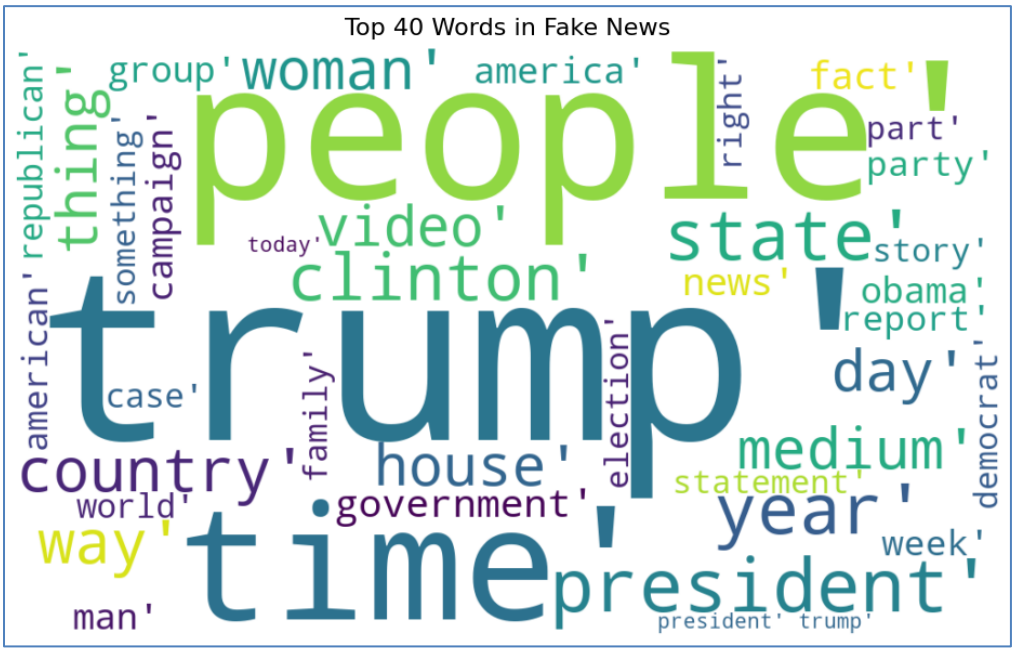
5.2.1 Find and display the top 40 words by frequency among true news in validation data after processing the text

- We have used a word cloud to find the top 40 words by frequency among true news in the validation data after processing the text.
- We have filtered news with label 1 (True News) and convert to it string and handle any non-string values.
- And then generated word cloud for True News.



5.2.2 Find and display the top 40 words by frequency among fake news in validation data after processing the text

- Similarly, we have used a word cloud to find the top 40 words by frequency among fake news in the validation data after processing the text.
- We have filtered news with label 0 (Fake News) and convert to it string and handle any non-string values.
- And then generated word cloud for Fake News.



5.3 Find and display the top unigrams, bigrams and trigrams by frequency in true news and fake news after processing the text

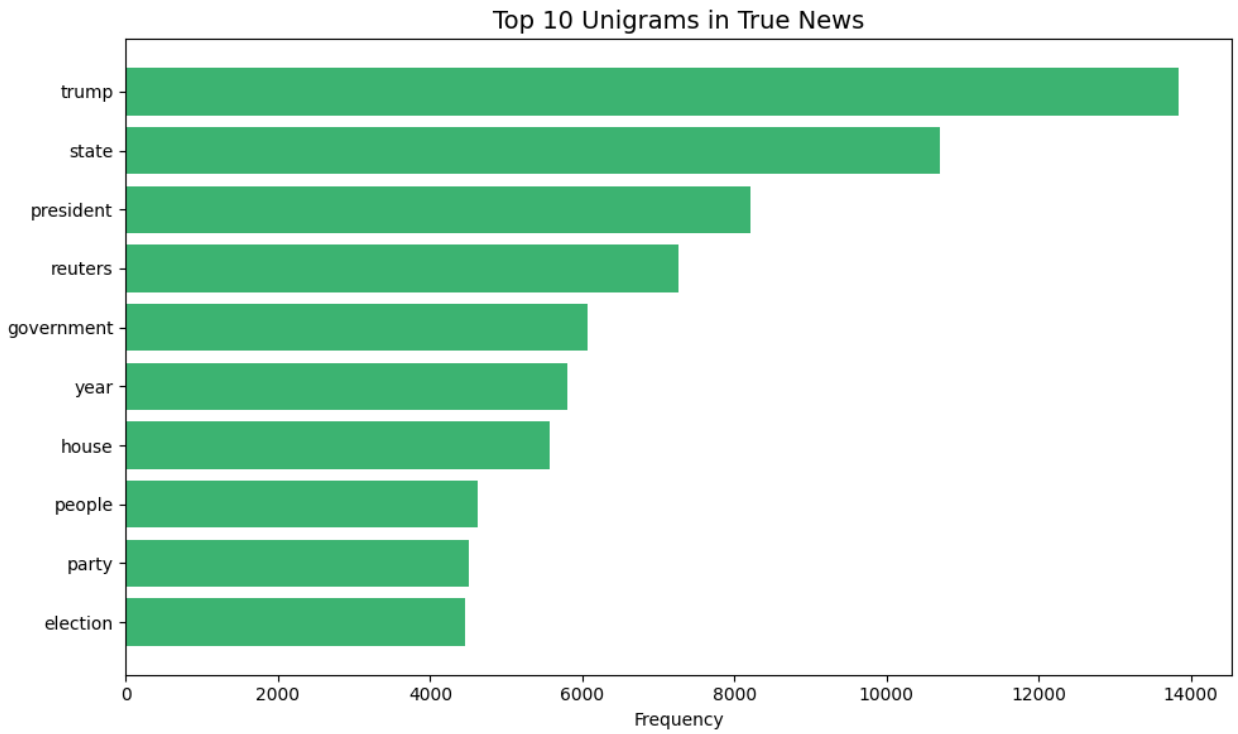
5.3.1 Write a function to get the specified top n-grams

- We have already defined a function in section 4.3.1 to get the specified top n-grams.
- This function takes corpus, n-gram range, how many top grams you need, title of the n-gram and color of the n-gram as input parameters.
- And plots n-gram for this.

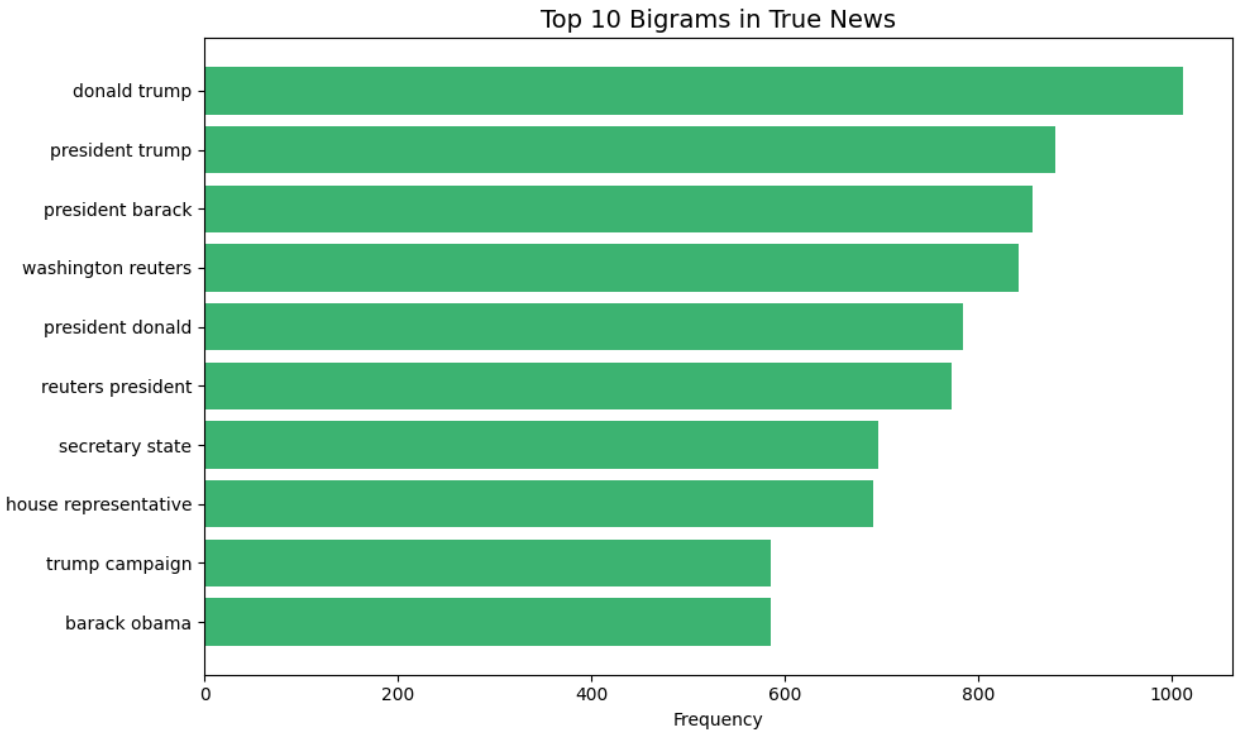
5.3.2 Handle the NaN values

- There are no NaN values present in validation dataframe as we have already handled them earlier.

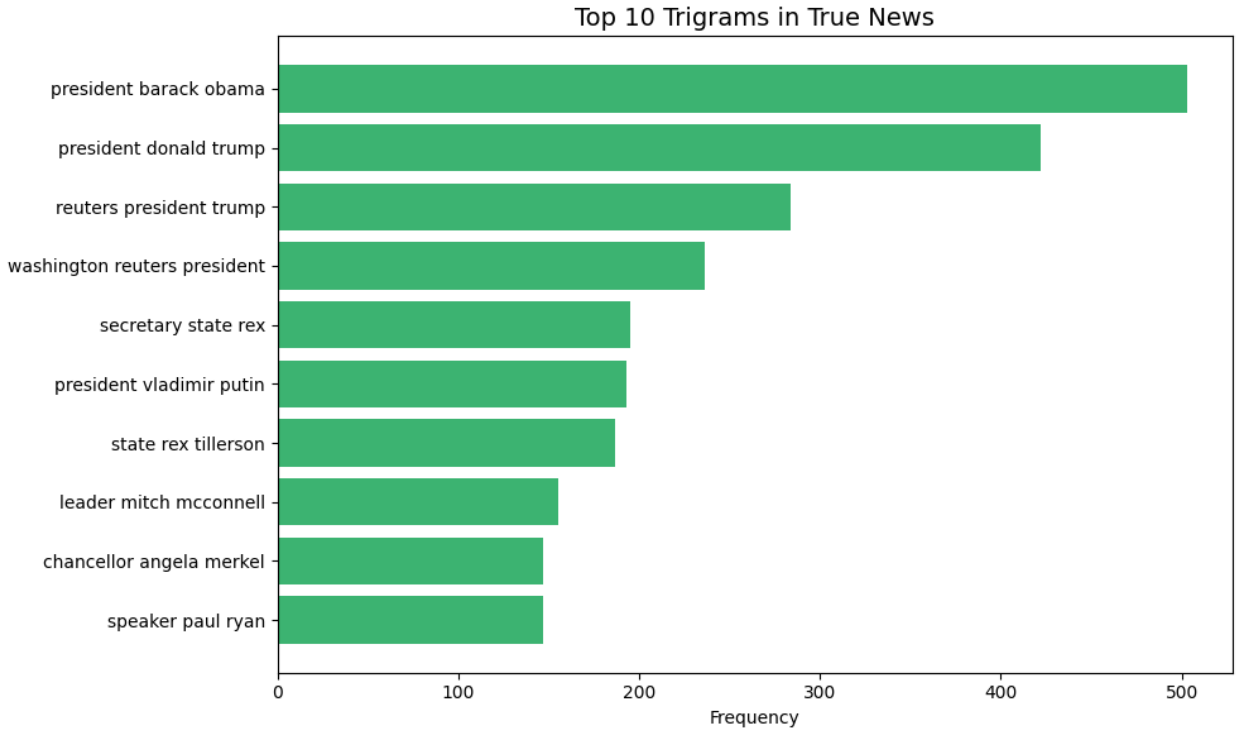
5.3.3 Display the top 10 unigrams by frequency in true news and plot them as a bar graph



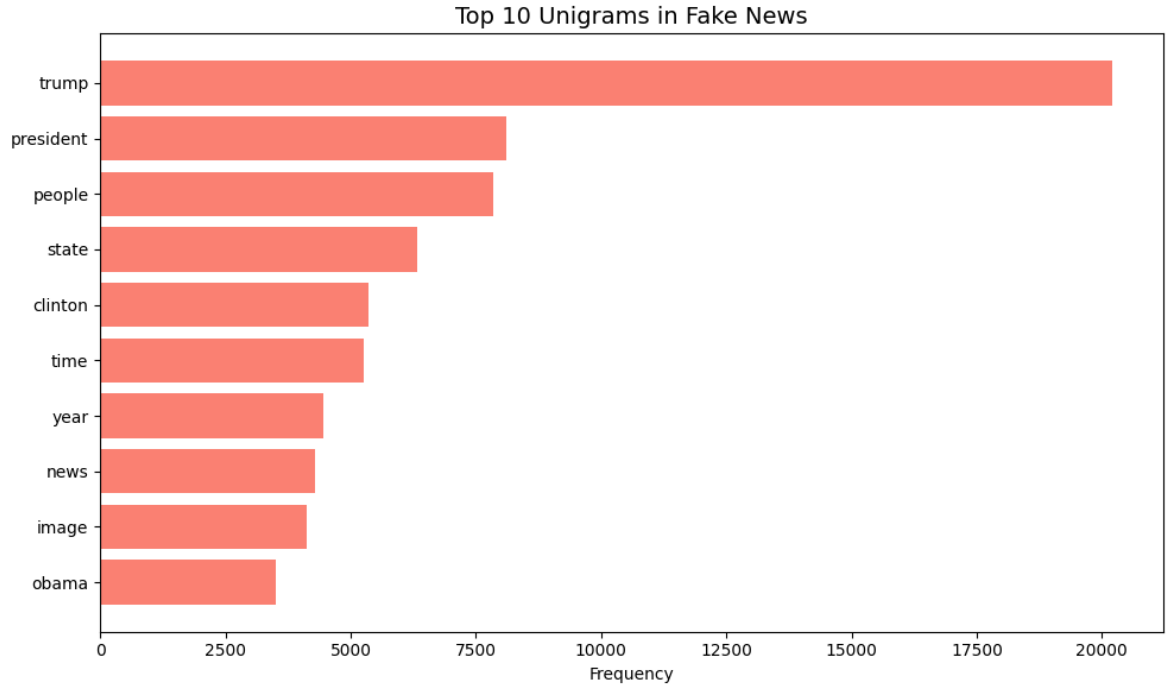
5.3.4 Display the top 10 bigrams by frequency in true news and plot them as a bar graph



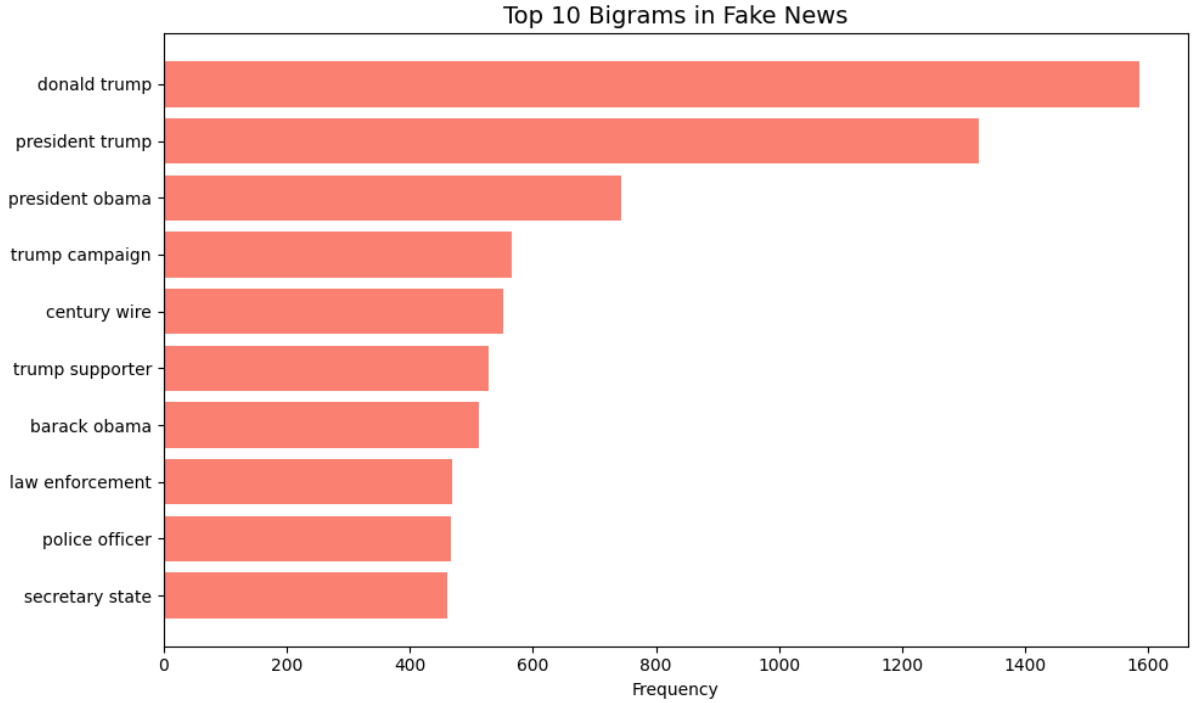
5.3.5 Display the top 10 trigrams by frequency in true news and plot them as a bar graph



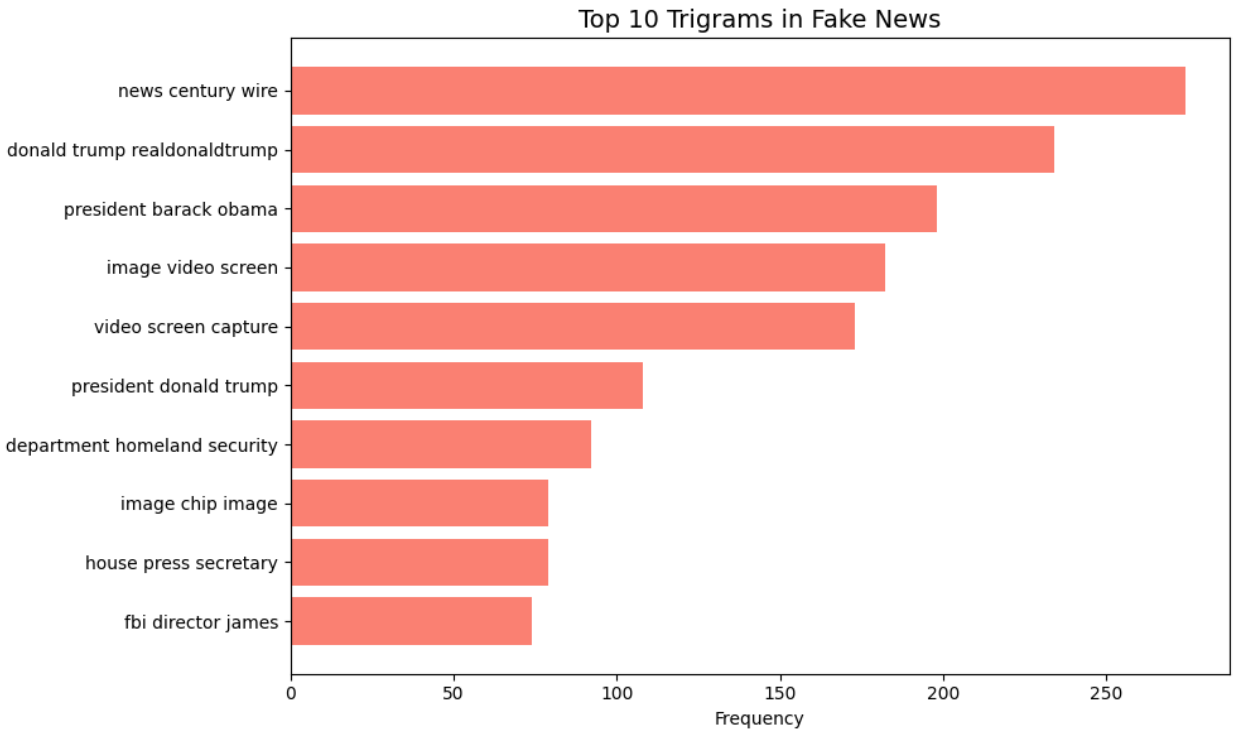
5.3.6 Display the top 10 unigrams by frequency in fake news and plot them as a bar graph



5.3.7 Display the top 10 bigrams by frequency in fake news and plot them as a bar graph



5.3.8 Display the top 10 trigrams by frequency in fake news and plot them as a bar graph



6. Feature Extraction

6.1 Initialise Word2Vec model

- We have initialized the Word2Vec model by downloading "word2vec-google-news-300".
- For any ML model to perform classification on textual data, we need to convert it to a vector form.
- In this assignment, we have used the Word2Vec Vectorizer to create vectors from textual data.
- Word2Vec model captures the semantic relationship between words.

6.2 Extract vectors for cleaned news data

- We have defined a function **get_doc_vector** to extract the vectors from the Word2Vec model.
- We have applied this function on both training and validation data to convert it to a vector form.

- And extracted the target variable for the training data and validation data.

```
X_train_vec shape: (31428, 300)
X_val_vec shape: (13470, 300)
y_train shape: (31428,)
y_val shape: (13470,)
```

7. Model Training and Evaluation

7.0 Import models and evaluation metrics

- We have imported models like LogisticRegression, DecisionTreeClassifier, RandomForestClassifier which we are going to use for classification problem.
- We have also imported the metrics like accuracy_score, precision_score, recall_score, f1_score to measure the performance of the models.
- We also imported the Classification report to check performance of the model.

7.1 Build Logistic Regression Model

7.1.1. Create and train logistic regression model on training data

- We have initialized Logistic Regression model with max iterations 1000.
- Once after initializing Logistic Regression model, we have trained it on training data.
- Then we have predicted labels on validation data.

7.1.2. Calculate and print accuracy, precision, recall and f1-score on validation data

- Based on true labels and predicted labels using Logistic Regression model, we have calculated accuracy, precision, recall, f1-score and classification report.

```
Accuracy : 0.7034
Precision: 0.6981
Recall   : 0.6703
F1-Score : 0.6839
```

```
              precision    recall  f1-score   support

           0       0.71      0.73      0.72      7022
           1       0.70      0.67      0.68      6448

    accuracy                           0.70     13470
   macro avg       0.70      0.70      0.70     13470
weighted avg       0.70      0.70      0.70     13470
```

7.2 Build Decision Tree Model

7.2.1 Create and train a decision tree model on training data

- We have initialized Decision Tree model with criterion as Gini, max depth of the tree as 35, min_sample_split as 4.
- Once after initializing Decision Tree model, we have trained it on training data.
- Then we have predicted labels on validation data.

7.2.2 Calculate and print accuracy, precision, recall and f1-score on validation data

- Based on true labels and predicted labels using Decision Tree model, we have calculated accuracy, precision, recall, f1-score and classification report.

```
Accuracy : 0.7084
Precision: 0.7098
Recall   : 0.6611
F1-Score : 0.6846
```

```
              precision    recall  f1-score   support

           0       0.71      0.75      0.73      7022
           1       0.71      0.66      0.68      6448

    accuracy                           0.71     13470
   macro avg       0.71      0.71      0.71     13470
weighted avg       0.71      0.71      0.71     13470
```

7.3 Build Random Forest Model

7.3.1 Create and train a random forest model on training data

- We have initialized Random Forest model with estimated trees as 200, max_depth as 35, min_samples_split=5, min_samples_leaf=3.
- Once after initializing Random Forest model, we have trained it on training data.
- Then we have predicted labels on validation data.

7.3.2 Calculate and print accuracy, precision, recall and f1-score on validation data

- Based on true labels and predicted labels using Random Forest model, we have calculated accuracy, precision, recall, f1-score and classification report.

```
Accuracy : 0.7934
Precision: 0.7980
Recall   : 0.7610
F1-Score : 0.7791
```

```
              precision    recall  f1-score   support

           0       0.79      0.82      0.81      7022
           1       0.80      0.76      0.78      6448

    accuracy                           0.79     13470
   macro avg       0.79      0.79      0.79     13470
weighted avg       0.79      0.79      0.79     13470
```

8. Conclusion

- This project demonstrates how **Machine Learning** can effectively tackle the challenge of fake news detection by leveraging **Word2Vec embeddings** and **Supervised Classification Models**.

Metric	Logistic Regression Model	Decision Tree Model	Random Forest Model
Accuracy	0.7034	0.7084	0.7934
Precision	0.6981	0.7098	0.7980
Recall	0.6703	0.6611	0.7610
F1-Score	0.6839	0.6846	0.7791

Key Insights

- The **original** and **cleaned** news texts have **similar character lengths**, but the **processed texts**—after lemmatization and stop-word removal—show a **marked reduction in length**, indicating effective text normalization.
- In both the **training** and **validation** sets, the words **"state"** and **"trump"** are among the most frequent, reflecting **election-related content**.
- The prominence of names like **"Obama"** and **"Trump"** suggests that the dataset is drawn from **U.S. news around the 2016 election**.
- **Unigrams, bigrams, and trigrams** effectively capture single words, two-word phrases, and three-word sequences, helping to uncover key patterns and context.

Model Comparison

- **Random Forest** delivers the **best performance across all key metrics**, achieving the highest **Accuracy (0.7934), Precision (0.7980), Recall (0.7610)**, and **F1-Score (0.7791)**, making it the most reliable choice for this task.
- - **Decision Tree** performs slightly better than Logistic Regression in **Accuracy** and **Precision**, but lags behind in **Recall**.
- - **Logistic Regression**, while easy to train and interpret, provides the lowest overall scores, indicating that linear decision boundaries are insufficient for this dataset.
- - Additional experiments also revealed that while **Naive Bayes** is great for quick baseline tests, the **Passive Aggressive Classifier** is often a strong

candidate for **real-world fake news detection systems**, thanks to its ability to handle large-scale online learning scenarios.

Takeaways

- This project successfully demonstrates the **complete NLP workflow**:
  1. **Text Preprocessing** – cleaning, lemmatization, stop-word removal.
  2. **Feature Extraction** – Word2Vec embeddings and n-gram analysis.
  3. **Model Training & Evaluation** – comparing multiple supervised models to identify the best performer.
- By mastering these steps, we've built a solid foundation for creating **intelligent systems** capable of identifying misinformation, contributing to a more **informed and trustworthy online environment**.